Nx Microfrontend Architecture with Module Federation

Production Deployment Blueprint for AKS (Shell + Employee Remote)

1. Overview of the Architecture

This architecture uses Nx + Angular + Module Federation to create a scalable microfrontend platform where:

- **Shell** is the host application deployed as an NGINX pod on AKS.
- **Employee** is a remote microfrontend deployed independently as its own NGINX pod.
- The shell loads the employee remote **lazily at runtime** using Module Federation.
- Both apps are built, versioned, and deployed independently.
- AKS provides isolation, scalability, and independent release cycles.

This pattern supports large enterprise Angular platforms with multiple teams and domains.

2. Nx Workspace Structure

apps/ shell/ → Host (Module Federation) employee/ → Remote (Module Federation) libs/ shared-ui/ shared-utils/ shared-data-access/

- Apps are deployable microfrontends.
- Libraries contain shared logic but are not part of the MFE boundary.
- Remotes are full Angular apps, not libraries.

3. Module Federation Configuration

Shell (Host)

`apps/shell/module-federation.config.js`

module.exports = { name: 'shell', remotes: [['employee', process.env.EMPLOYEE_REMOTE_URL],], };

Employee (Remote)

`apps/employee/module-federation.config.js`

module.exports = { name: 'employee', exposes: { './Module': 'apps/employee/src/app/remote-entry/entry.module.ts', }, };

4. Lazy Loading the Employee Remote

The shell loads the employee remote only when the user navigates to `/employee`.

`shell/src/app/app.routes.ts`

import { loadRemoteModule } from '@nx/module-federation'; export const appRoutes: Routes = [{ path: 'employee', loadChildren: () => loadRemoteModule('employee', './Module').then(m => m.RemoteEntryModule), },];

This ensures:

- No employee code is bundled into the shell.
- The remote is fetched via HTTP at runtime.
- The shell and employee remain independently deployable.

5. Build Commands

Shell

nx build shell --configuration=production

Employee

nx build employee --configuration=production

Output:

dist/apps/shell/ dist/apps/employee/

6. Dockerfiles

Shell Dockerfile

FROM node:20-alpine AS builder WORKDIR /app COPY . . RUN npm install RUN npx nx build shell --configuration=production FROM nginx:1.25-alpine COPY dist/apps/shell/ /usr/share/nginx/html EXPOSE 80 CMD ["nginx", "-g", "daemon off;"]

Employee Dockerfile

FROM node:20-alpine AS builder WORKDIR /app COPY . . RUN npm install RUN npx nx build employee --configuration=production FROM nginx:1.25-alpine COPY dist/apps/employee/ /usr/share/nginx/html EXPOSE 80 CMD ["nginx", "-g", "daemon off;"]

7. AKS Deployment Blueprint

7.1 Shell Deployment

`shell-deployment.yaml`

apiVersion: apps/v1 kind: Deployment metadata: name: shell spec: replicas: 3 selector: matchLabels: app: shell template: metadata: labels: app: shell spec: containers: - name: shell image: .azurecr.io/shell: ports: - containerPort: 80 env: - name: EMPLOYEE_REMOTE_URL value: "https://employee.mycompany.com/remoteEntry.js"

7.2 Shell Service

apiVersion: v1 kind: Service metadata: name: shell-service spec: type: ClusterIP selector: app: shell ports: - port: 80 targetPort: 80

7.3 Shell Ingress

apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: shell-ingress spec: rules: - host: shell.mycompany.com http: paths: - path: / pathType: Prefix backend: service: name: shell-service port: number: 80

7.4 Employee Deployment

`employee-deployment.yaml`

apiVersion: apps/v1 kind: Deployment metadata: name: employee spec: replicas: 3 selector: matchLabels: app: employee template: metadata: labels: app: employee spec: containers: - name: employee image: .azurecr.io/employee: ports: - containerPort: 80

7.5 Employee Service

apiVersion: v1 kind: Service metadata: name: employee-service spec: type: ClusterIP selector: app: employee ports: - port: 80 targetPort: 80

7.6 Employee Ingress

apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: employee-ingress spec: rules: - host: employee.mycompany.com http: paths: - path: / pathType: Prefix backend: service: name: employee-service port: number: 80

8. Runtime Flow Diagram

User → shell.mycompany.com | | (initial load) ↓ Shell Angular App | | (navigate to /employee) ↓ loadRemoteModule() | | (HTTP request) ↓ employee.mycompany.com/remoteEntry.js | ↓ Employee Remote Module Loaded

9. CI/CD Strategy

Shell Pipeline

- Install dependencies
- `nx affected:build`
- Docker build
- Push to ACR
- Deploy to AKS

Employee Pipeline

- Same steps, independent
- Deployment does not require shell redeploy

This enables independent release cycles.

10. Key Advantages

- Independent deployments
- Lazy loading of remotes
- Zero coupling between shell and employee
- Runtime composition via Module Federation
- Scalable for large enterprise Angular platforms
- AKS provides isolation, autoscaling, and resilience

Concepts :-

Nx for Angular is essentially a **smart, opinionated monorepo build system** that gives Angular teams a way to scale codebases, enforce architecture, and accelerate development with tooling that goes far beyond the Angular CLI. The core ideas revolve around **workspace structure, project graph intelligence, computation caching, generators, executors, and modular architecture patterns**.

What Nx Angular Really Is

Nx extends the Angular CLI with **workspace-level intelligence**. Instead of treating each Angular app as an isolated project, Nx treats your entire codebase as a **graph of interconnected libraries and applications**, then optimizes everything around that graph.

The result is:

- Faster builds and tests
- Enforced architecture boundaries
- Reusable libraries
- Consistent scaffolding
- Better CI/CD performance
- A scalable monorepo strategy

Core Concepts You Need to Learn

1️⃣ Workspace & Project Graph

Nx analyzes your code and builds a **project graph**:

- Nodes = apps and libraries
- Edges = dependencies between them

This graph powers:

- Affected commands (only rebuild what changed)
- Enforced boundaries
- Visual dependency maps
- Smarter CI pipelines

Understanding the graph is foundational because **everything else in Nx builds on it**.

2️⃣ Libraries (Feature, UI, Data-Access, Utility)

Nx encourages breaking your Angular code into **domain-driven libraries**:

- **Feature libraries** — screens, flows, business logic
- **UI libraries** — components, design system
- **Data-access libraries** — services, API clients, NgRx slices

- **Utility libraries** — pure functions, helpers

Why this matters:

- Reuse
- Isolation
- Faster builds
- Clear ownership
- Enforced boundaries

This is the heart of Nx's architecture philosophy.

3️⃣ Enforced Module Boundaries

Nx uses **tags** and **lint rules** to enforce architecture rules:

- Feature libs can't import other features
- UI libs can't import data-access
- Shared libs can't depend on domain libs

This prevents "spaghetti imports" and keeps your architecture clean over time.

4️⃣ Generators (Code Scaffolding)

Nx provides generators that create:

- Angular apps
- Angular libraries
- Components
- Services
- NgRx slices
- Storybook setups
- Jest test configs

Generators ensure:

- Consistency
- Best practices
- Correct folder structure

- Automatic wiring into the workspace

You can also create **custom generators** for your organization.

5️⃣ Executors (Build, Serve, Test, Lint)

Executors are Nx's version of Angular CLI builders:

- Build Angular apps
- Serve dev servers
- Run tests (Jest, Karma, Cypress)
- Lint code
- Run custom scripts

Executors can be:

- Provided by Nx
- Provided by Angular
- Custom-built

They integrate with caching and the project graph.

6️⃣ Computation Caching (Local + Distributed)

Nx caches:

- Builds
- Tests
- Lint results
- Any executor output

If nothing changed, Nx **replays the result instantly**.

Distributed caching (Nx Cloud) allows:

- CI to reuse developer cache
- Developers to reuse CI cache

This is one of Nx's biggest performance wins.

7️⃣ Affected Commands

Nx can detect what changed and run tasks **only for impacted projects**:

- `nx affected:build`
- `nx affected:test`
- `nx affected:lint`

This is powered by the project graph + git diff.

Huge for CI/CD efficiency.

8️⃣ Task Orchestration

Nx runs tasks in the correct order based on dependencies:

- Parallel where possible
- Sequential where required
- Cached where unchanged

This makes Nx ideal for large monorepos with many apps.

9️⃣ Standalone vs Integrated Angular Projects

Nx supports:

- **Standalone Angular apps** (Angular 15+)
- **Module-based Angular apps**

Nx generators automatically adapt to whichever style you choose.

🔟 Nx Dev Tools (VSCode + Graph UI)

Nx provides:

- Visual project graph

- Task runner UI
- Dependency insights
- Workspace analysis

This makes it easier to understand and maintain large Angular codebases.

How These Concepts Fit Together (Mental Model)

Nx Concept	Why It Matters	What It Enables
Project Graph	Understand dependencies	Affected builds, orchestration
Libraries	Modular architecture	Reuse, boundaries
Boundaries	Prevent bad imports	Long-term maintainability
Generators	Consistency	Faster onboarding
Executors	Build/test/lint	CI/CD automation
Caching	Speed	Instant rebuilds
Affected Commands	Efficiency	Scalable pipelines
Task Runner	Correct ordering	Parallel execution

Nx Cloud	Distributed cache	Team-wide speedups

What to Learn First (Recommended Path)

1. **Nx Workspace Basics**
   Apps vs libs, folder structure, project graph.

2. **Library Architecture**
   Feature/UI/Data-access patterns.

3. **Generators**
   Create apps, libs, components the Nx way.

4. **Boundaries & Tags**
   Enforce clean architecture.

5. **Caching & Affected Commands**
   Build/test only what changed.

6. **Task Orchestration**
   Understand how Nx runs tasks in parallel.

7. **Nx Cloud (optional)**
   Distributed caching for teams.

A Non-Obvious Insight

Nx is not just a monorepo tool — it's an **architecture enforcement engine**.
The real power is not in faster builds but in **preventing architectural decay** as your Angular codebase grows.

If you want, I can map this into a **step-by-step learning roadmap**, or create a **visual architecture diagram** showing how Nx organizes Angular apps and libraries.

A complete Nx–Angular learning roadmap works best when it mirrors how Nx itself is designed: **from workspace fundamentals → architecture → tooling → optimization → enterprise patterns**. This gives you a mental model that scales from a single Angular app to a multi-domain, multi-team monorepo.

---

1. Foundations of an Nx Workspace

Understanding the workspace is the anchor for everything else. Nx treats your codebase as a **graph of projects** rather than a flat folder of files.

Key ideas

- A workspace contains **apps** (deployable) and **libs** (reusable).
- Nx analyzes imports to build a **project graph**.
- Every command (build, test, lint) runs with graph awareness.

What to learn

- Workspace structure (`apps/`, `libs/`)
- Project configuration (`project.json`)
- Root-level configuration (`nx.json`, `tsconfig.base.json`)
- How Nx extends Angular CLI

---

2. The Project Graph and Dependency Intelligence

The project graph is Nx's superpower. It lets Nx understand:

- Which libraries depend on which others
- Which apps depend on which libraries
- What is affected when a file changes

Why it matters

- Enables **affected** commands
- Enables **task orchestration**
- Enables **architecture enforcement**
- Enables **caching**

What to learn

- How to visualize the graph (`nx graph`)
- How Nx computes dependencies
- How to interpret circular dependency warnings

---

3. Library-Driven Architecture (Nx's Angular Philosophy)

Nx encourages a **modular, domain-driven architecture** using libraries.

Library types you should master

- **Feature libraries** — screens, flows, business logic
- **UI libraries** — components, design system
- **Data-access libraries** — API clients, NgRx slices, services
- **Utility libraries** — pure functions, helpers
- **Shell libraries** — routing, layout, app-level wiring

Why this matters

- Enforces separation of concerns
- Enables reuse
- Reduces build/test scope
- Makes large Angular systems maintainable

---

4. Enforced Boundaries and Tags

Nx uses **tags** + **ESLint rules** to enforce architecture rules.

What to learn

- How to tag libraries (`tags: ['domain:payments', 'type:ui']`)

- How to define constraints (`only import from…`)
- How to prevent cross-domain imports
- How to prevent feature-to-feature imports

Why it matters

This prevents architecture drift and keeps your monorepo clean over years.

---

5. Generators (Scaffolding)

Generators create consistent, standardized code.

What to learn

- Create Angular apps (`nx g @nx/angular:app`)
- Create Angular libraries (`nx g @nx/angular:lib`)
- Create components, services, routes
- Create NgRx slices
- Create Storybook setups
- Create custom generators for your org

Why it matters

- Eliminates boilerplate
- Enforces patterns
- Speeds up onboarding
- Ensures consistency across teams

---

6. Executors (Build, Serve, Test, Lint)

Executors are Nx's task runners.

What to learn

- Build Angular apps with Nx executors
- Serve apps with dev servers
- Run Jest/Karma tests

- Run Cypress/Playwright e2e tests
- Create custom executors

Why it matters

Executors integrate with caching, orchestration, and the project graph.

---

7. Computation Caching (Local + Distributed)

Nx caches the output of:

- Builds
- Tests
- Lint
- Any executor

What to learn

- How local caching works
- How distributed caching works (Nx Cloud)
- How to inspect cache hits/misses
- How to make custom tasks cacheable

Why it matters

This is where Nx gives **10x faster CI/CD** and **instant rebuilds**.

---

8. Affected Commands and Smart CI

Nx can detect what changed and run only what's impacted.

What to learn

- `nx affected:build`
- `nx affected:test`
- `nx affected:lint`
- How Nx uses git diff

- How to build CI pipelines around affected commands

Why it matters

This is the key to scaling Angular monorepos without blowing up CI times.

---

9. Task Orchestration and Parallel Execution

Nx runs tasks:

- In parallel when possible
- In sequence when required
- With caching when unchanged

What to learn

- How Nx determines task order
- How to visualize task execution
- How to optimize task pipelines
- How to define target dependencies

Why it matters

This is how Nx handles dozens of apps and hundreds of libraries efficiently.

---

10. Standalone Angular + Nx

Angular is moving toward **standalone components**. Nx supports both:

- Module-based Angular
- Standalone Angular

What to learn

- How Nx generators adapt to standalone
- How routing works in standalone mode
- How to structure libraries in standalone Angular

---

11. Nx Dev Tools and Workspace Visualization

Nx provides:

- Visual project graph
- Task runner UI
- Workspace analysis
- Dependency insights

What to learn

- How to use the Nx Console VSCode extension
- How to inspect dependency issues
- How to explore the workspace visually

---

12. Enterprise Patterns with Nx + Angular

Once you understand the fundamentals, the next layer is enterprise architecture.

What to learn

- Domain-driven design in Angular monorepos
- Cross-team library ownership
- Versioning and release strategies
- API layering and data-access patterns
- NgRx architecture in Nx workspaces
- Microfrontends with Module Federation + Nx

---

13. Advanced Nx Customization

For deep mastery, learn how to extend Nx itself.

What to learn

- Custom generators

- Custom executors
- Custom plugins
- Workspace-specific schematics
- Organization-wide architecture rules

Summary Table: What to Learn and Why It Matters

Area	What You Learn	Why It Matters
Workspace	Structure, config	Foundation for everything
Project Graph	Dependencies	Affected builds, orchestration
Libraries	Feature/UI/Data-access	Modular architecture
Boundaries	Tags, constraints	Prevent architecture drift
Generators	Scaffolding	Consistency, speed
Executors	Build/test/lint	CI/CD integration
Caching	Local + distributed	Massive performance gains
Affected Commands	Smart CI	Only run what changed
Orchestration	Parallel tasks	Scale to large monorepos
Standalone Angular	Modern Angular	Future-proof architecture
Nx Dev Tools	Visualization	Debugging + insights
Enterprise Patterns	DDD, ownership	Multi-team scaling
Customization	Plugins, rules	Org-wide standardization

A natural next step is choosing **how deep you want to go**: do you want a hands-on learning plan with exercises, or a conceptual architecture map showing how Nx organizes Angular domains?